How-To Library

Informatica

# Configure ETL scanners to use information about files that match a pattern

© Copyright Informatica LLC 2022. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

# Abstract

Effective in version 10.5.3, you can improve lineage from jobs that use wildcard characters such as asterisks (*) and question marks (?) to access files. This article explains how to prepare input and configure scanners when you have incomplete lineage because of wildcards.

The steps apply to the Databricks Notebooks and Microsoft Azure Data Factory scanners that extract metadata from Amazon S3 and Azure Data Lake Storage (ADLS).

# Supported Versions

- Enterprise Data Catalog 10.5.3

# Table of Contents

Use case. . . . . 2
Solution. . . . . 3
How to automatically use information about files that match a pattern. . . . . 3
How to manually use information about files that match a pattern. . . . . 5

# Use case

A large company owns substantial data processing systems that access directory paths and file names with the following wildcard characters:

- An asterisk (*) substitutes multiple characters
- A question mark (?) substitutes a single character

The files are typically in remote file systems such as Amazon S3 or Azure Data Lake Storage (ADLS).

The company uses Databricks Notebooks and Microsoft Azure Data Factory scanners to extract metadata from Amazon S3 and ADLS file systems. The jobs generate incomplete lineage because of wildcards used in directory paths and file names.

The challenge is to point to the missing information about the files that contain wildcards when you configure the scanner in the MetaDex tool, as shown in the following examples:

## Amazon S3 sample directory

The s3://transactions/aggregates/ directory contains the following files:

- orders_2020.csv
- orders_2021.csv
- orders_2022.csv

The scanner job loads all files that match s3://transactions/aggregates/orders_*.csv into the database table ORDERS. To extract complete lineage, the scanner needs the complete file names.

## Azure Data Lake Storage sample directory

The abfss://container@storage.dfs.core.windows.net/aggregates/ directory contains the following files:

- orders_2020.csv
- orders_2021.csv

- orders_2022.csv

The scanner job loads all files that match abfss://container@storage.dfs.core.windows.net/aggregates/orders_*.csv into the database table ORDERS. To extract complete lineage, the scanner needs the complete file names.

# Solution

You can choose one of the following methods to get wildcard lineage:

**Automatic scanning from the Catalog**

Scan the applicable Amazon S3 buckets or ADLS containers to the resource in the Catalog that contains the scanned Amazon S3 or ADLS location. For example: s3://transactions/aggregates/

This method is fast, simple, and requires less updates. Select this method if your files are already scanned to a Catalog resource or you plan to scan them in the future.

**Manual scanning from Files.csv**

Create a Files.csv file in the connection registry. Files.csv supplies additional file definitions, such as file names and columns.

This method is tedious and complex as it requires you to identify and write files definitions manually. You need to update the file each time you change the file definitions. However, you may want to choose this method if you have many buckets or containers that you can't load into the Catalog.

The Files.csv file must contain two columns with the FilePath and ColumnName headers. The FilePath column specifies the complete path to the file including the file name. The ColumnName column specifies a column in the file.

For example, the Files.csv file under mdrepo/transactions might contain the following rows:

```
"FilePath","ColumnName"
"/aggregates/orders_2020.csv", "OrderNumber"
"/aggregates/orders_2020.csv", "OrderTotal"
"/aggregates/orders_2021.csv", "OrderNumber"
"/aggregates/orders_2021.csv", "OrderTotal"
"/aggregates/orders_2022.csv", "OrderNumber"
"/aggregates/orders_2022.csv", "OrderTotal"
```

# How to automatically use information about files that match a pattern

You can use an Amazon S3 or a Microsoft Azure Data Lake resource to extract the metadata that you need to resolve the wildcards.

## Step 1. Create and run a MetaDex configuration

In the MetaDex tool, create a configuration to scan metadata and run it once. For more information, see "Creating a configuration" and "Running a scanner job" in the *MetaDex Scanner Configuration Guide*.

## Step 2. Create an Enterprise Data Catalog resource

In the Catalog, create one of the following resources to scan files that include wildcards in the file metadata:

- Amazon S3. For more information, see "Amazon S3" in the *Enterprise Data Catalog Scanner Configuration Guide*.
- Azure Data Lake Storage. For more information, see "Microsoft Azure Data Lake Storage" in the *Enterprise Data Catalog Scanner Configuration Guide*.

## Step 3. Update the connection registry

A MetaDex scanner job cannot identify the required resource in the Catalog without an entry in the connection registry. Add the resource name to the MetaDex connection registry to ensure that the scanner job can access the resource.

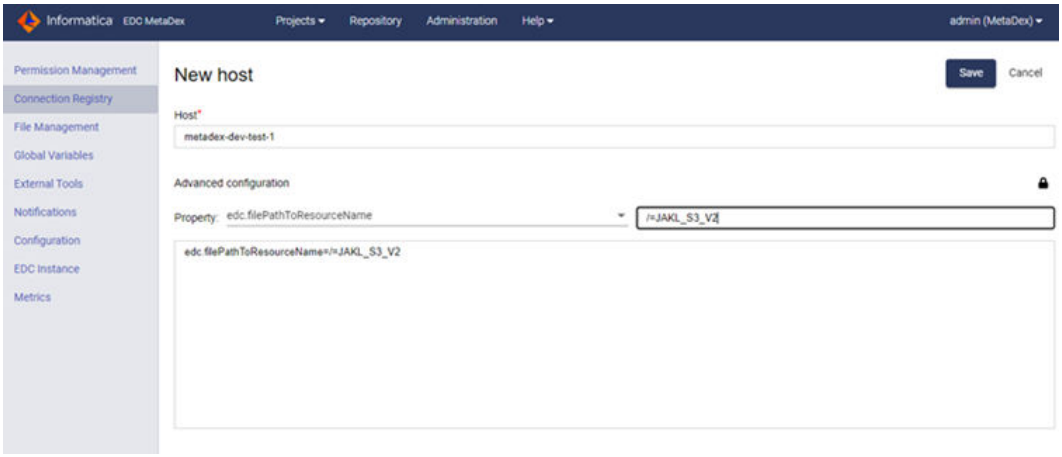
For more information, see "Connection registry" in the *MetaDex Scanner Administrator Guide*.

Perform the following steps to provide the resource name in the MetaDex connection registry when you create a connection:

1. In the MetaDex tool, on the header, click **Administration**.
2. On the left pane, click **Connection Registry**.

   The Connection Registry entries list is displayed in the right pane.
3. Click **Add host**.

   The **New host** window appears.
4. Enter the new host name, as shown in the following example:




   **Note:** The host is the MetaDex connection registry host from which the CSV call reads the file. For example:

   - If a job reads the file from the s3a://ACCESS_KEY:SECRET_KEY@metadex-dev-test-1/demo4/*input.csv URL, the MetaDex connection registry host name is: `metadex-dev-test-1`
   - If a job reads the file from the abfss://container@storage.dfs.core.windows.net/aggregates/orders_*.csv URL, the host name is: `storage.dfs.core.windows.net`
5. In the **Property** field, enter the value in one of the following formats:
   - /=<RESOURCE_NAME> maps the bucket or the container.
   - /<path_to_the_scanned_directory>=<RESOURCE_NAME> maps a part of the bucket or a container.

6. Click **Save**.

   The **Host** window appears, as shown in the following image:




   Where:

   metadex-dev-test-1 is the MetaDex connection registry host name.

   metadex-dev-test-1 is also the name of the bucket or the container.

   /=JAKL_S3_V2 maps the bucket or a container represented by the MetaDex connection registry host to the resource that you created in the previous step.

   JAKL_S3_V2 is the resource name.

### Step 4. Run the configuration job

Run the configuration job again. For more information, see "Running a scanner job" in the *MetaDex Scanner Configuration Guide*.

### Step 5. Verify the resource

Perform the following steps to verify the resource in the Catalog:

1. Reopen the resource.

   The file name is correctly resolved, for example, orders_2022.csv instead orders_*.csv, and it has all columns. The lineage between the input and the output file is also correct. The input and output, however, are still created as reference elements.

2. In the **Connection Assignment** workspace, connect the file system resource with the job resource. For more information, see "Managing Connections" and "Assigning Connections to Reference Resources" in the *Catalog Administrator Guide*.

   The Connection Assignment workspace updates. After a couple of minutes, the updated lineage appears without wildcards and with file names resolved based on the content of the resource.

## How to manually use information about files that match a pattern

You can extract file names that match a pattern from a CSV file.

### Step 1. Create and run a MetaDex configuration

In the MetaDex tool, create a configuration to scan metadata and run it once. For more information, see "Creating a configuration" and "Running a scanner job" in the *MetaDex Scanner Configuration Guide*.

### Step 2. Create and modify Files.csv

Perform the following steps to provide the missing file names and data structures:

1. Create a Files.csv configuration file, as shown in the following example:

```
"FilePath","ColumnName"
"/aggregates/orders_2020.csv", "OrderNumber"
"/aggregates/orders_2020.csv", "OrderTotal"
"/aggregates/orders_2021.csv", "OrderNumber"
"/aggregates/orders_2021.csv", "OrderTotal"
"/aggregates/orders_2022.csv", "OrderNumber"
"/aggregates/orders_2022.csv", "OrderTotal"
```

   **Note:** Add the files from the input code that contain wildcards as FilePath entries.

2. In the ${SCANNERS_WORKSPACE}/MDREPO directory, create a subdirectory. The name of the subdirectory must indicate the host of the source files. For example: `aggregates`

   **Tip:** You can find the host name in MetaDex debug log files, as shown in the following example:

```
22/10/17 20:33:07.122 [main] DEBUG
(LoggingFileDefinitionRepository:59) - Searching for many files by pattern:
AmazonS3FileLocationInfo[bucketName=aggregates,linkedServiceName=AmazonS31,origin=REMOTE,
path=/aggregates/*.csv,host= aggregates,id=<null>]
```

### Step 3. Upload the Files.csv file

Upload the Files.csv configuration file to the subdirectory that you created in the previous step. You can create more than one directory that represents a host. Each directory can contain a Files.csv file with a list of files that are available for a given host, as shown in the following example:

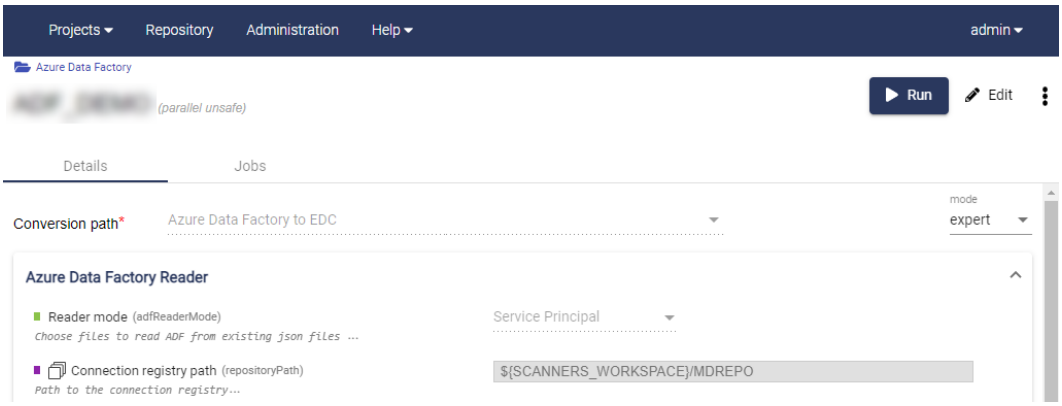
```
- MDREPO
- aggregates
- Files.csv
```

Where aggregates is a subdirectory that contains the Files.csv file.

### Step 4. Update the connection registry

Perform the following steps to update the connection registry parameter:

1. Open the configuration.
2. Change the user mode to **expert**.
3. In the reader module of your configuration, in the **Connection Registry path** field, enter the path to the connection registry directory that contains the file directories, as shown in the following example:




### Step 4. Run the configuration job

Run the configuration job again. For more information, see "Running a scanner job" in the *MetaDex Scanner Configuration Guide*.

### Step 6. Verify the object structure and resource

On the **Generated objects** tab, verify if the object structure is complete.

## Authors

**Katarzyna Strzalkowska**

**The author would like to acknowledge Rashmi Mani, Divya Maheshvary, Tomasz Duszka, Dawid Duda, Wojciech Kijas, and Ewa Pydych for help with this article.**